

Brandforge AI: A Multi-Agent System for Automated Brand Asset Generation

Tianrui Fang
Columbia University
New York, NY, USA
tf2639@columbia.edu

Hao Chen
Columbia University
New York, NY, USA
hc3625@columbia.edu

Tianyu Zhan
Columbia University
New York, NY, USA
tz2704@columbia.edu

Abstract

Early-stage startups struggle to create professional brand assets due to limited design and marketing resources. We present Brandforge AI, a multi-agent system that generates landing pages, logos, and marketing strategies from a single natural-language description using a three-stage pipeline: a Brand Agent extracts a structured profile; an AI Gateway dispatches three specialized agents in parallel; and a React frontend renders outputs.

The Marketing Agent uses a ReAct-style loop with Retrieval-Augmented Generation (RAG) over ad policies and benchmarks for grounded strategies. The Logo Agent generates SVG and DALL-E 3 logos with post-hoc palette enforcement, and the UI/Web Agent produces responsive HTML/Tailwind pages via plan-then-generate with deterministic backfill.

All agents use fallback mechanisms to ensure valid outputs. We evaluate UI completeness, latency, and output quality, and discuss trade-offs in LLM-based generation.

Keywords: multi-agent systems, RAG, LLM, automated brand design, SVG generation

1 Introduction

Early-stage startups and small businesses face a persistent challenge: building a coherent brand identity requires professional assets—a landing page, a logo, and a marketing strategy—yet most founders lack the expertise, time, or budget to produce them. Freelancers and agencies are costly and slow, while template-based tools (e.g., Wix, Canva) produce generic, non-adaptive results.

Large language models (LLMs) have shown strong performance in creative generation, from code to visual design. However, producing a *coherent* set of brand assets remains challenging, as outputs must share a consistent identity while satisfying different structural constraints across HTML, SVG, and natural language.

We present **Brandforge AI**, a multi-agent system that generates a complete brand identity from a single natural-language description. Given an textual input, the system produces three artifacts in parallel:

- A **responsive landing page** in HTML/Tailwind CSS with semantic sections,
- A **vector logo** (SVG) and raster logo (DALL-E 3), both constrained to a shared color palette,

- A **marketing strategy** with platform-specific recommendations grounded via Retrieval-Augmented Generation (RAG) over ad policies and benchmark data.

The system is built around four design principles: (1) **structured extraction**, where a BrandProfile is derived from free text and shared across agents; (2) **parallel dispatch**, where an AI Gateway executes agents concurrently via `asyncio.gather`; (3) **graceful degradation**, where deterministic fallbacks ensure partial outputs under failure; and (4) **color consistency**, where theme colors propagate through all agents using modality-specific enforcement strategies.

Brandforge AI is implemented as a full-stack system with a Python/FastAPI backend, a React/TypeScript frontend, MongoDB persistence, and OpenAI GPT-4o and DALL-E 3 for generation. The codebase and evaluation are publicly available, along with a demo of the end-to-end workflow.

Project repository: https://github.com/JarrettChen217/EECS-E6895-Big_Data_AI-Brandforge-AI

Demo video: https://drive.google.com/file/d/1WnP9LV0BYa1d_jqvEly1MAXnqRhV-PET/view

2 Related Work

2.1 LLMs for Creative Generation

Large language models [1] and multimodal extensions enable AI-driven creative tools. GPT-4o [9] supports structured JSON, SVG, and HTML/CSS generation, while DALL-E 3 [8] produces high-quality images from text. Prior work on LLM-based design spans web design [12], logo generation, and copywriting, but typically treats each artifact independently. Brandforge AI instead generates a *coherent set* of assets with shared identity, enforced via structured contracts and validation.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [5] improves factuality by retrieving relevant context before generation. Our Marketing Agent uses sentence-transformers [10] and FAISS [4] to retrieve ad policies (Meta, Google, TikTok, LinkedIn) and benchmark data, grounding outputs in real constraints.

2.3 Multi-Agent Architectures

Multi-agent systems improve reliability by decomposing tasks. The ReAct framework [15] motivates our Marketing

Agent’s reasoning-and-tool pipeline, while chain-of-thought prompting [13] informs the UI/Web Agent’s plan-then-generate design, where a structured outline constrains HTML generation.

2.4 Automated Brand Asset Tools

Template-based tools (Canva, Wix) require manual customization, while AI logo generators (Looka, Hatchful) produce isolated assets without consistency across outputs. Recent tools (Galileo AI, Canva Magic Design, Wix) reduce effort via natural language but remain single-artifact focused.

Brandforge AI instead uses a shared BrandProfile to coordinate a multi-agent system, enforcing consistency across logo, website, and marketing outputs.

3 System Design

3.1 Overview

Brandforge AI follows a three-tier pipeline architecture, illustrated in Figure 1:

1. **Brand Extraction:** A free-text brand description is processed by the Brand Agent into a structured BrandProfile (brand name, primary/accent colors, font family, tone, industry, tagline).
2. **Parallel Generation:** An AI Gateway receives the BrandProfile and dispatches sub-agents—Marketing, Logo, and UI/Web—in parallel via `asyncio.gather`. Each agent is an independent async task.
3. **Result Aggregation:** Agent outputs are collected and persisted to MongoDB; the frontend polls for completion and renders the results.

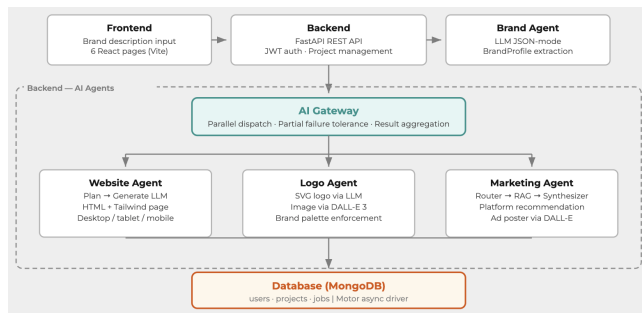


Figure 1. System architecture of Brandforge AI. The Brand Agent extracts a structured BrandProfile from free-text input. The AI Gateway fans out to three specialized agents (Marketing, Logo, UI/Web) in parallel. Results are aggregated and persisted to MongoDB for frontend rendering.

3.2 Brand Agent

The Brand Agent preserves user intent while conservatively filling missing fields such as tone, target audience, and tagline. Colors are constrained to explicit hex values; invalid outputs are replaced with style-specific defaults, unless a Brand

Hub palette overrides them. This structured profile ensures consistent interpretation across all downstream agents.

The Brand Agent is the first stage of the pipeline. It takes a free-text brand description and a style preference and calls GPT-4o with `response_format={"type": "json_object"}` to produce a structured BrandProfile. The extracted profile contains:

- **brand_name:** Inferred from user input.
- **primary_color, accent_color:** Hex color codes representing the brand’s visual identity.
- **font_family:** Recommended web font.
- **tone:** The brand’s communication tone.
- **industry:** The inferred industry category.
- **tagline:** An optional generated slogan.

Color extraction uses a multi-layer fallback strategy: LLM JSON is validated with Pydantic, and colors are coerced into valid hex format. If validation fails, the agent falls back to style-specific defaults

3.3 AI Gateway

The Gateway serves as a fault-isolation boundary. Each agent returns structured output with status metadata, allowing partial success; a failed logo generation, for instance, does not block SVG, web, or marketing outputs.

This is important for LLM systems due to potential failures, timeouts, or invalid formatting. Decoupling orchestration from generation also improves extensibility, enabling future agents (e.g., social media, SEO, ad planning) without pipeline changes.

The Gateway acts as a fan-out dispatcher: it receives a BrandProfile and requested outputs, then runs agents in parallel via an AGENT_REGISTRY. The `dispatch()` function creates `asyncio.Task` objects and executes them with `asyncio.gather`, tolerating partial failures by returning error states per agent while continuing execution.

3.4 Marketing Agent

The Marketing Agent is the most architecturally complex agent in the system. It implements a ReAct-style reasoning loop [15] with three stages:

1. **Route:** An LLM router classifies the user’s marketing question and produces a JSON execution plan—a sequence of tool calls selected from the agent’s tool registry.
2. **Execute:** The plan is executed sequentially. Each step invokes a tool (e.g., the RAG tool for ad policy retrieval, or the platform chooser for benchmark analysis) and feeds the output into the next step’s context.
3. **Synthesize:** A synthesizer LLM merges all tool outputs into a coherent final marketing strategy with inline source citations.

The execution plan is capped at five RAG steps to bound latency. If the router LLM fails, a deterministic fallback plan

is used: platform chooser followed by three RAG queries covering general, visual, and textual ad policies.

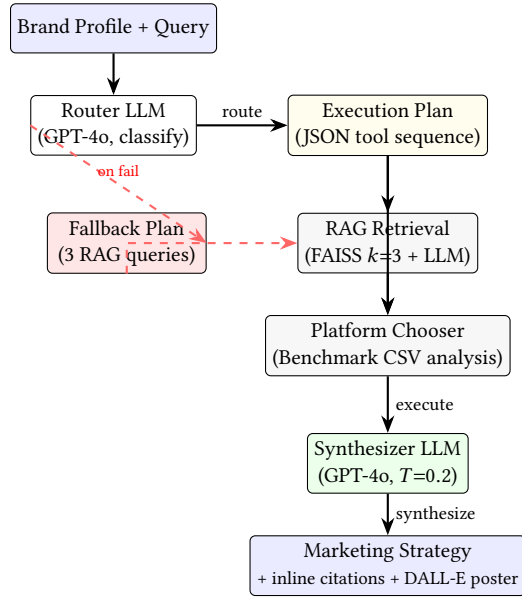


Figure 2. Marketing Agent pipeline. The Router classifies the query and returns a JSON execution plan. The Executor runs RAG retrieval and platform benchmark analysis sequentially. The Synthesizer merges results into a cited strategy, with a deterministic fallback plan on router failure.

3.4.1 RAG Pipeline. The RAG pipeline retrieves relevant ad policy documents to ground marketing recommendations:

- **Corpus:** 18 curated JSONL documents covering ad policies from Meta, Google Ads, TikTok, and LinkedIn. Each document contains a `doc_id`, platform label, title, and full text.
- **Embedding:** `sentence-transformers/all-MiniLM-L6-v2` [10] encodes both queries and documents into 384-dimensional vectors.
- **Retrieval:** FAISS [4] performs approximate nearest-neighbor search with $k = 3$.
- **Generation:** Retrieved documents and their `doc_id` references are injected into the LLM prompt; the LLM generates a grounded answer with source citations.

3.4.2 Platform Chooser. The platform chooser tool loads benchmark CSV data covering Meta (2024 Q3 by industry) and Google Ads (2025 by industry), including metrics such as CPC, CPM, CPA, CTR, and conversion rate. It also incorporates global and US-specific audience behavior data. For a given brand, the tool filters benchmarks by industry using token-overlap matching and returns a structured recommendation comparing platform suitability, cost efficiency, and audience alignment.

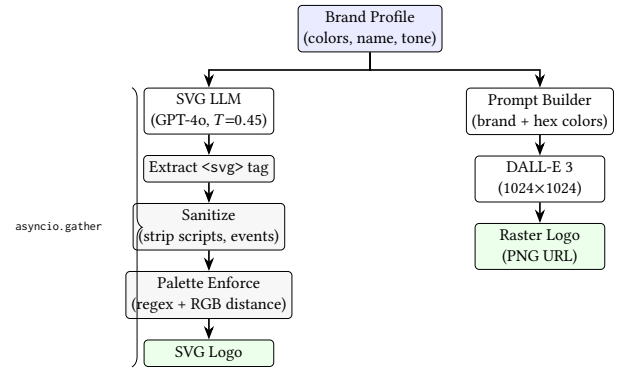


Figure 3. Logo Agent pipeline. SVG and DALL-E 3 generation run in parallel with `asyncio.gather`. The SVG path applies extraction, sanitization, and palette enforcement, while the DALL-E path uses brand identity fields. Either mode can fail without blocking the other.

3.5 Logo Agent

The Logo Agent generates brand logos in two complementary modalities (see Figure 3):

GPT-4o produces SVG via a constrained prompt requiring palette-only colors, verbatim wordmark rendering, and a single composition. The output is then post-processed:

1. **Extraction:** The `<svg>` block is extracted from markdown or raw output.
2. **Sanitization:** Scripts, event handlers, and unsafe attributes are removed; a default `viewBox` is added if missing.
3. **Palette Enforcement:** A regex-based pass replaces non-palette colors in fill, stroke, stop-color, and inline styles with the nearest allowed palette color.

In parallel, a DALL-E 3 prompt derived from the brand profile generates a raster logo at 1024×1024 resolution. Both SVG and raster generation run concurrently via `asyncio.gather` in a thread pool, and failures in either mode return empty outputs instead of propagating errors.

3.6 UI/Web Agent

The UI/Web Agent generates a complete, responsive HTML/Tailwind CSS landing page using a two-step plan-then-generate pipeline.

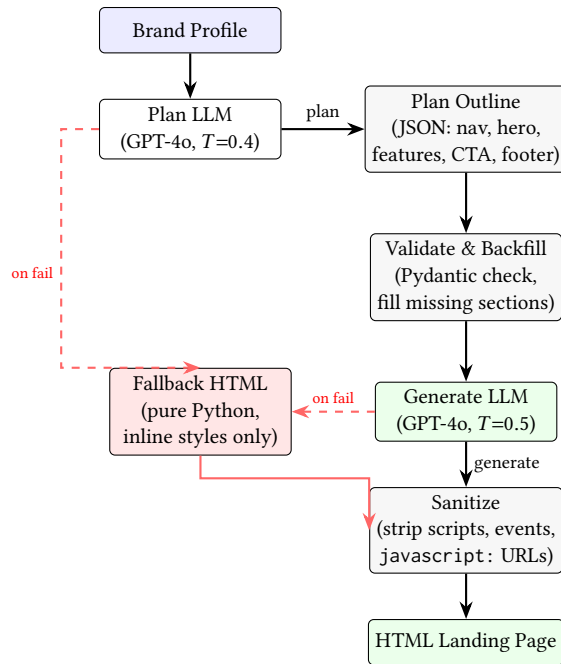


Figure 4. UI/Web Agent pipeline. The Plan LLM produces a structured JSON outline; missing sections are backfilled deterministically. The Generate LLM produces HTML from the validated plan. On any LLM failure, a pure-Python fallback produces a clean hero page with no external dependencies.

3.7 UI/Web Agent

The UI/Web Agent generates a responsive HTML/Tailwind CSS landing page using a two-step plan-then-generate pipeline.

3.7.1 Plan Step. The first LLM call produces a structured JSON outline (PlanOutline) for five semantic sections:

- **Nav:** Brand name and navigation links.
- **Hero:** Headline, sub-headline, CTA text, and background description.
- **Features:** Three feature cards with titles and descriptions.
- **CTA:** Final call-to-action headline and button text.
- **Footer:** Brand name, copyright, and placeholder links.

The prompt enforces JSON-only output, tone-aware copy, and section word limits. After parsing, `_validate_or_backfill` checks each section with Pydantic models and **backfills** missing or invalid sections using deterministic defaults from the brand profile. This preserves valid LLM output while filling gaps without a full fallback.

3.7.2 Generate Step. The second LLM call converts the validated outline into a complete HTML document. The prompt enforces eight constraints:

1. Output standalone HTML only.
2. Include the Tailwind CSS CDN script.
3. Include a Google Fonts link for the selected font.
4. Use the brand’s primary and accent colors.

5. Render all five sections using the plan content verbatim.
6. Avoid broken image assets.
7. Ensure responsive layout with Tailwind utilities.
8. Adapt the visual style to the brand tone.

3.7.3 Sanitization and Fallback. Generated HTML is sanitized by removing `<script>` tags except Tailwind CDN, inline event handlers (`on*=`), and `javascript:` URLs. If either LLM call fails or output is unparseable, the agent calls `build_fallback_html()`, a deterministic Python fallback that creates a full-viewport hero page with the brand name, tagline, and CTA. It uses inline styles only and computes readable text color from the brand’s primary color.

3.8 Frontend Architecture

The frontend exposes the multi-agent pipeline through a single brand workspace while separating outputs into distinct panels for review and download. This improves usability by keeping a unified brand context while allowing independent inspection of each artifact.

It is built with React 18, TypeScript, Vite 6, and Tailwind CSS 4, and provides four main views via sidebar navigation:

- **Brand Hub:** Full pipeline interface for brand input, palette selection (predefined or custom), tone selection, and output toggles. Results are shown in tabbed panels with live previews (SVG/PNG logo, iframe-rendered website, and Markdown marketing strategy).
- **Web Architect:** Standalone website generator with color, font, and device preview controls.
- **Logo Lab:** Standalone logo generator with SVG/PNG export.
- **Marketing Chat:** Chat-based interface for marketing strategy with Markdown rendering.

State is persisted via a custom `usePersistedState` hook using `localStorage`. The frontend uses a typed API client with JWT authentication. Project creation triggers a background pipeline, and the UI polls `GET /api/projects/{id}` every 2 seconds (with a 180-second timeout), displaying per-agent loading states during generation.

4 Implementation and Data

4.1 Backend Infrastructure

A project document stores the original prompt, style options, extracted `BrandProfile`, generation status, timestamps, and final outputs. Job documents track per-agent execution state, including agent name, status, output payload, and errors, enabling both full-project retrieval and fine-grained debugging.

Since OpenAI SDK calls are synchronous, LLM execution is wrapped in executor-based async calls to avoid blocking FastAPI’s event loop and maintain API responsiveness.

The backend uses FastAPI with a factory pattern via `create_app()`. A lifespan context manager initializes MongoDB on startup and cleans up on shutdown, with CORS configured for Vite origins. The API is organized into three routers:

- **Auth Router:** POST `/api/auth/token` exchanges an API key for a JWT, with a dev-key bypass for local use.
- **Projects Router:** POST `/api/projects` triggers the pipeline; GET `/api/projects/{id}` polls results; GET `/api/projects` lists user projects.
- **Agents Router:** Direct access to agents via `/agents/brand/profile`, `/agents/marketing`, `/agents/logo`, and `/agents/website`.

4.2 Data Sources

4.2.1 Ad Policy Corpus (RAG). JSONL corpus contains ad policy documents from major platforms:

- **Meta:** Prohibited content, restricted goods and services, policy principles, checklist, personal attributes, health claims, and financial ads. [7]
- **Google Ads:** Prohibited content and practices, restricted content, and editorial/technical requirements. [3]
- **TikTok:** General policies, creative guidelines, health/finance policies, and ad quality requirements. [11]
- **LinkedIn:** Prohibited and restricted content. [6]

4.2.2 Advertising Benchmarks. Six CSV files provide platform advertising performance benchmarks:

- `Meta_2024Q3_By_Industry.csv`: CPC, CPM, CPA, CTR, and conversion rate by industry for Meta Ads.
- `Google_Ads_2025_By_Industry.csv`: Equivalent metrics for Google Ads in the US market.
- `CPC_CPM_CPA_{Global_Average,US_Only}.csv`: Cross-platform cost averages.
- `Audience_Behavior_{Global_Average,US_Only}.csv`: Platform-level audience metrics, including age distribution, mobile usage, and purchase intent.

These benchmarks come from public industry reports (WordStream, Digitopia, HubSpot, DataReportal, Search Engine Journal). The platform chooser loads them at query time and filters by industry using token-overlap matching. [14] [2]

4.3 Marketing Agent Implementation

The Marketing Agent was migrated from the midterm Flask prototype to FastAPI, preserving the route $\rightarrow$ plan $\rightarrow$ synthesize pipeline while updating HTTP handling and LLM initialization.

LLM Abstraction: A `BaseLLM` interface exposes `generate(messages, max_new_tokens, temperature)`, with `OpenAILLM` wrapping the OpenAI chat API. A singleton factory (`get_llm()`) lazily initializes the model and avoids global state issues.

Tool Registry: Tools are managed via a `ToolRegistry`, where each tool defines a name, description, and `async __call__`. The RAG tool is created via `make_tool_rag()`, binding FAISS retrieval and the LLM in a closure. Benchmarks are loaded from disk at runtime using `BENCHMARKS_DIR`.

Synthesizer: The synthesizer combines execution traces and RAG citations into a brand-conditioned prompt (name, industry, tone). Generation runs at temperature 0.2 with up to 1024 tokens. If citations are missing, a fallback source line is appended.

DALL-E Poster Generation: After strategy synthesis, the agent generates an ad poster via DALL-E 3 using a prompt derived from the brand profile and key strategy signals.

4.4 Logo Agent Implementation

SVG Generation: The LLM is called with temperature 0.45 and `max_tokens=2048`. The system prompt is intentionally prescriptive, listing allowed colors explicitly and requiring the brand wordmark. Post-hoc palette enforcement uses a regex-based approach:

- SVG attributes (`fill`, `stroke`, `stop-color`, `color`) are matched and rewritten.
- Inline CSS style declarations are parsed for color properties and similarly rewritten.
- Colors are matched to the closest allowed palette color by Euclidean distance in RGB space.

DALL-E Generation: The prompt builder concatenates brand identity fields (name, industry, tone) with the allowed hex colors into a structured prompt.

4.5 UI/Web Agent Implementation

Plan Generation: A JSON-only LLM call (temperature 0.4, `max_tokens=600`) produces the page structure. `_validate_or_backfill()` enforces a Pydantic schema and fills missing sections using brand-derived defaults (e.g., `hero = brand name + "Welcome"`, `features = "Quality"`, `"Innovation"`, `"Reliability"`).

HTML Generation: A second LLM call (temperature 0.5, `max_tokens=3000`) generates full HTML under strict prompt constraints. Output is extracted via `extract_html()` and sanitized via `sanitize_html()`, removing scripts and event handlers.

Fallback HTML: `build_fallback_html()` is a standalone Python function that generates a minimal HTML page with inline styles. Text color is chosen using a luminance heuristic: dark text for light backgrounds and white text otherwise.

4.6 Testing

The test suite focuses on deterministic behavior in failure-prone components. Unit tests cover fallback HTML generation, backfill, code extraction, SVG validation, sanitization, event-handler removal, empty API-key handling, and LLM failure fallback. The UI/Web Agent includes 12 `pytest`/`pytest-asyncio` tests with mocked LLMs for deterministic execution.

Integration testing uses a Postman collection and a Bash script (`test_website_e2e.sh`) to verify full pipeline execution and structured outputs.

While not a user study, the tests cover key reliability properties: correct formatting, safe rendering, partial failure handling, and end-to-end generation.

5 Evaluation and Results

5.1 Evaluation Methodology

We evaluate Brandforge AI along four dimensions:

1. **UI Completeness:** Binary check for presence of required landing page sections (nav, hero, features, CTA, footer).
2. **Generation Speed:** End-to-end latency from project creation to all agent outputs, measured in seconds, with a 30s target enabled by parallel execution.
3. **Visual Satisfaction:** Internal 1–5 Likert rating of color consistency, layout coherence, and aesthetic quality.
4. **User Rating:** External user feedback on relevance, professionalism, and usefulness for startup branding.

5.2 Results

Table 1. Evaluation results from internal testing.

Metric	Target	Result
UI Completeness	100% (5/5 sections)	100% in tested runs
Generation Speed	< 30 s	Usually within target
Visual Satisfaction	≥ 4.0/5.0	Positive internal ratings
User Rating	≥ 4.0/5.0	Requires broader study

5.3 Qualitative Observations

Outputs were most coherent when prompts clearly specified industry, audience, and tone. The shared BrandProfile improved consistency across logo, landing page, and marketing strategy. The UI/Web Agent reliably produced complete pages, and the Marketing Agent generated practical, policy-grounded recommendations.

Quality issues were most evident in visual generation. SVG logos were more controllable due to post-hoc color rewriting, while DALL-E 3 outputs only approximately matched palettes. Landing page aesthetics also varied across runs due to inherent LLM non-determinism.

5.4 Test Case Example: Automotive Startup Advertising

We tested Brandforge AI with the following prompt:

VoltWay is a compact electric car brand for young professionals in large cities. The tone should be clean, modern, trustworthy, and environmentally conscious.

The Brand Agent inferred an electric mobility brand targeting urban professionals with a clean, sustainability-focused tone and a blue-green palette. The Logo Agent produced a minimal mobility-themed identity, and the UI/Web Agent generated a landing page emphasizing affordability, compact design, parking convenience, and charging access. The Marketing Agent recommended short-form video and search ads focused on cost-effective urban commuting.

A representative ad concept was: “Drive the city differently. VoltWay gives you the freedom of an electric car without the cost or stress of ownership.” This demonstrates coordinated generation of visual, web, and marketing assets from a single description.

6 Discussion

6.1 Limitations

LLM Dependency and Non-Determinism: All agents rely on GPT-4o and DALL-E 3, causing output variability even at low temperature. API limits, outages, and fragile prompts can fail generation, while fallbacks are lower quality.

Color Consistency Across Modalities: Cross-modal palette enforcement is imperfect. SVG regex handling misses formats such as `rgba()`, `hsl()`, and `currentColor`; HTML relies on prompts; and DALL-E 3 only approximates exact colors.

Evaluation Scope: Evaluation is small-scale and qualitative. Stronger validation would include visual regression testing, larger A/B user studies, and quantitative palette adherence metrics.

Limited Domain Coverage: The ad policy corpus covers four platforms and 18 documents, and benchmark data may become outdated. Broader, continuously updated sources are needed.

6.2 Insights

Structured Intermediates Improve Reliability: The BrandProfile is the system’s core contract. Extracting it early and sharing it across agents reduces ambiguity and improves modality-specific generation. The UI/Web Agent’s plan step similarly improves reliability over single-shot HTML generation.

Defense in Depth for LLM Outputs: Prompting alone is insufficient. Regex-based color enforcement catches invalid Logo Agent outputs, while the UI/Web Agent’s backfill preserves valid structure and fills missing content.

Graceful Degradation Enables Partial Progress: Allowing partial failure across agents, gateway, and frontend improves robustness. Users still receive valid outputs even if one agent fails.

RAG Grounding Improves Output Trustworthiness: RAG grounds the Marketing Agent in ad policies and benchmarks, making recommendations more specific, constrained, and metric-aware.

References

- [1] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33 (2020).
- [2] Digitopia. [n. d.]. Aggregated anonymous data from Q3 2024 across Meta Platforms (Facebook, Instagram). <https://digitopialb.com/benchmarks>. Accessed: 2026-03-3.
- [3] Google. [n. d.]. Google Ads policies. <https://support.google.com/adspolicy/answer/6008942?hl=en>. Accessed: 2026-03-8.
- [4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3, 535–547.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.
- [6] LinkedIn. [n. d.]. LinkedIn Advertising Policies. <https://www.linkedin.com/legal/ads-policy>. Accessed: 2026-03-8.
- [7] Meta. [n. d.]. Introduction to the Advertising Standards. <https://transparency.meta.com/policies/ad-standards/>. Accessed: 2026-03-8.
- [8] OpenAI. 2023. DALL-E 3 System Card. *OpenAI Technical Reports* (2023).
- [9] OpenAI. 2024. GPT-4o System Card. *OpenAI Technical Reports* (2024).
- [10] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
- [11] TikTok. [n. d.]. Ad format and functionality. <https://ads.tiktok.com/help/article/tiktok-ads-policy-ad-format-and-functionality>. Accessed: 2026-03-8.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.
- [14] WordStream. [n. d.]. Google Ads Benchmarks 2025: Competitive Data Insights for Every Industry. <https://www.wordstream.com/blog/2025-google-ads-benchmarks>. Accessed: 2026-03-3.
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.